Software Implementation and Testing Document

For

Fantasy Football

Version 1.0

Authors:
Miles G
Tanner S
Edward P
Amadou A

1. Programming Languages (5 points)

The project utilizes Python for the backend API and data processing services, React with TypeScript for the frontend user interface, and SQL (PostgreSQL) for database management. Python was selected for its rich ecosystem, backend web frameworks, and ease of handling scheduled stats caching services. In Python, we leverage the SQLAlchemy and Supabase libraries to simplify database interaction and management, with Supabase serving as our free cloud PostgreSQL solution. React and TypeScript were chosen for the frontend to construct a dynamic, type-safe, and responsive single-page application. PostgreSQL is used as the relational database to persistently store static player metadata, user accounts, league structures, roster lineups, and facility levels.

2. Platforms, APIs, Databases, and other technologies used (5 points)

Our application currently relies on our own custom REST APIs, using Vite to connect the React/TypeScript frontend to the Python Flask backend. We use SQLAlchemy as the ORM to execute database commands and manage data models, with Supabase serving as our free PostgreSQL database hosting solution. While full integrations with the external Tank01 NFL API and Supabase Auth are planned for future increments, current stats and authentication workflows operate through our local API endpoints and database schema.

3. Execution-based Functional Testing (10 points)

Execution-based functional testing was conducted manually by verifying end-to-end user workflows against functional requirements; no automated test code was written for this process. Key test suites verified user registration/authentication, role-based access control for League Managers, and manual roster assignment. Functional testing focused on verifying the communication between the Flask backend and test data, as well as Vite connecting the React frontend to the backend to successfully reach and display data. Scoreboard precision and live-updating weekly matchup standings were also tested against defined endpoints.

4. Execution-based Non-Functional Testing (10 points)

Execution-based non-functional testing focused on payload validation and local test environment authentication bypass settings. Requests sent to backend API endpoints were checked for proper JSON formatting and required parameter handling. In addition, authentication bypass settings for local test environments were verified to secure production endpoints while supporting seamless developer testing.

5. Non-Execution-based Testing (10 points)

Non-execution-based testing consisted of structured team code reviews, walk-throughs, and schema design inspections. The team reviewed UML class diagrams, sequence diagrams, and API specifications to ensure consistency across backend endpoints and frontend components. Peer reviews were conducted to verify database schema relations (such as roster entries linking real-life NFL players across multiple leagues) and to validate requirement coverage across all system modules prior to implementation.